09
자연어 처리 기초

문장은 숫자가 아닙니다. 문장을 잘게 자르고, 다듬고, 번호를 붙여 신경망이 받을 수 있는 같은 길이의 숫자 줄로 바꾸는 전처리를, 정의부터 세우고 영화 리뷰에 적용해 봅니다.

오늘 다루는 것

1. 토큰화: 단어, 문장, 한국어 형태소
2. 텍스트 데이터의 정제와 정규화
3. 불용어
4. 정규 표현식: 문법과 모듈 함수
5. 전처리 실습: 단어 집합, 정수 인코딩, 패딩

끝나고 나면 할 수 있는 것

1. 토큰, 형태소, 불용어, 단어 집합, 패딩을 정의하고 각각이 왜 필요한지 설명할 수 있다.
2. 정규 표현식 패턴을 기호 하나씩 뜯어 읽을 수 있다.
3. 리뷰 문장들을 (문장 수, 최대 길이) 모양의 정수 텐서로 직접 바꿀 수 있다.

2026년 10월 8일 목요일 14:30 (10/8 자료 1/2)

발표자 설명, 코랩 실습, 퀴즈 돌아가며 풀기 순서로 진행합니다. 코랩에서는 먼저 `!pip install konlpy` 를 실행해 주세요.

09

자연어 처리 기초

07장의 Char RNN은 글자를 번호로, 번호를 원-핫 벡터로 바꿔 넣었습니다. 실제 자연어 처리에서는 글자보다 큰 단위(단어)를 쓰고, 데이터도 깨끗하지 않습니다. "이 영화 정말 재밌어요!! ㅋㅋ" 같은 문장을 신경망에 넣으려면 먼저 **잘게 자르고, 다듬고, 번호를 붙여** 같은 길이의 숫자 줄로 만들어야 합니다. 이 과정을 전처리라고 합니다. 어떻게 전처리했느냐에 따라 모델이 보는 데이터 자체가 달라집니다.

왜 배우나: 오늘의 학습 경로



개념마다 먼저 정의를 세우고, **영화 리뷰를 긍정/부정으로 나누는 모델에 넣을 데이터 만들기**라는 상황에 적용합니다. 09-01~09-04에서 도구를 하나씩 익히고, 09-05에서 리뷰 6개를 처음부터 끝까지 텐서로 바꿉니다. 이 텐서가 12장의 임베딩, 13장의 텍스트 분류 모델의 입력이 됩니다.

09-01 토큰화

핵심 한 줄 **토큰화는 글을 의미 있는 단위(토큰)로 자르는 일이고, 영어는 띄어쓰기와 규칙으로 대부분 되지만 한국어는 형태소 단위로 잘라야 한다.**

왜 배우나 신경망은 문장을 통째로 이해하지 못합니다. 작은 조각으로 나눠야 조각마다 번호를 붙이고(09-05) 벡터로 바꿀 수 있습니다(12장). 어디서 자르느냐에 따라 "서로 다른 단어가 몇 개인가"가 달라지고, 이것이 모델의 크기와 성능을 좌우합니다. 전처리의 첫 단추입니다.

1) 토큰과 토큰화

정의

- 말뭉치(corpus, 코퍼스): 분석하려고 모아 둔 글 전체. 리뷰 20만 개를 모았다면 그것이 말뭉치입니다.
- 토큰(token): 글을 나눈 단위 조각. 보통 단어이고, 필요에 따라 문장, 형태소, 글자가 될 수도 있습니다.
- 토큰화(tokenization): 말뭉치를 토큰으로 나누는 작업
  - 단어 토큰화: 토큰의 기준이 단어
  - 문장 토큰화: 토큰의 기준이 문장. 말뭉치가 문장으로 나뉘어 있지 않을 때 먼저 합니다.

띄어쓰기나 구두점으로 자르기만 하면 될 것 같지만, 그렇게 하면 안 되는 경우가 있습니다. 토큰화할 때 정해야 하는 것들입니다.

- **구두점과 특수 문자를 무조건 지우면 안 된다.** 마침표는 문장의 끝을 알려 주고, `Ph.D.` , `$45.55` , `01/02/06` 처럼 기호가 단어의 일부인 경우가 있습니다.
- **줄임말과, 띄어쓰기가 들어 있는 단어.** `what're` 는 `what are` 의 줄임말이고(접어), `New York` , `rock 'n' roll` 은 띄어쓰기가 있어도 한 단어입니다.
- **정답은 없고 규칙이 있다.** 도구마다 규칙이 다릅니다. 널리 쓰이는 Penn Treebank 규칙은 하이픈으로 이어진 단어(`home-based`)는 하나로 두고, 아포스트로피로 붙은 접어(`doesn't`)는 `does` 와 `n't` 로 나눕니다.

예시 적용 영어 문장 "Don't be fooled by the dark sounding name, Mr. Jone's Orphanage."를 NLTK의 두 도구로 잘라 봤습니다(실습 1).

도구	`Don't` 를 자른 결과	규칙
`word_tokenize`	`Do` / `n't`	접어 n't를 뜻이 있는 조각으로 분리 (Penn Treebank 방식)
`WordPunctTokenizer`	`Don` / `'` / `t`	구두점을 모두 별도 토큰으로 분리

문장 토큰화에서도 같은 문제가 생깁니다. "I am looking for Ph.D. students. and you are a Ph.D. student."를 마침표마다 자르면 "Ph."에서 끊깁니다. NLTK의 `sent_tokenize` 는 Ph.D.의 마침표를 문장 끝으로 보지 않고 두 문장으로 바르게 나눴습니다.

2) 한국어 토큰화와 형태소

정의

- 어절: 띄어쓰기 단위. "영화를", "연휴에는"이 각각 한 어절입니다.
- 교착어: 단어 뒤에 조사나 어미가 붙어 문법적 역할을 나타내는 언어. 한국어가 교착어입니다. "그가, 그에게, 그를, 그와, 그는"은 모두 같은 "그"에 조사만 다릅니다.
- 형태소(morpheme): 뜻을 가진 가장 작은 말의 단위
  - **자립 형태소**: 혼자서도 단어가 되는 것. 명사, 대명사, 부사 등 ("영화", "정말")
  - **의존 형태소**: 다른 형태소에 붙어야 쓰이는 것. 조사, 어미, 접사, 어간 ("가", "를", "-었-")

- 형태소 분석기: 어절을 형태소로 나눠 주는 도구. 파이썬에서는 KoNLPy 패키지의 Okt, 꼬꼬마(Kkma), Mecab 등을 씁니다.
- 품사 태깅(POS tagging): 토큰마다 명사, 조사, 동사 같은 품사를 붙이는 것. 같은 글자라도 품사에 따라 뜻이 달라지기 때문에 필요합니다 (영어 fly는 동사 "날다"와 명사 "파리", 한국어 "못"은 명사와 부사).

왜 한국어는 띄어쓰기로 자르면 안 될까? 첫째, 어절로 자르면 같은 단어가 조사마다 다른 토큰이 되어 버립니다. 둘째, 한국어는 띄어쓰기를 지키지 않아도 읽을 수 있어서 실제 글에서는 띄어쓰기가 자주 틀립니다. 영어는 띄어쓰기를 빼면 읽기 어렵습니다.

예시 적용

리뷰 네 개 "영화가 정말 재밌다", "영화를 또 보고 싶다", "영화는 지루했다", "영화의 음악이 좋다"는 모두 "영화"에 대한 글입니다(실습 2). 띄어쓰기로 자르면 "영화가", "영화를", "영화는", "영화의"라는 **서로 다른 토큰 4종류**가 되어, 모델은 이들이 같은 단어라는 것을 모릅니다. 형태소로 자르면 토큰 "영화"가 4번 나온 것으로 셉니다.



그림 1 어절로 자르면 조사마다 다른 토큰이 되지만, 형태소로 자르면 같은 단어가 같은 토큰으로 모인다

이 작은 예에서 서로 다른 토큰의 수는 띄어쓰기 12개, 형태소 14개로 형태소 쪽이 오히려 많습니다. 조사 "가, 를, 는, 의, 이"가 토큰으로 따로 생겼기 때문입니다. 하지만 리뷰가 수만 개로 늘면 조사의 종류는 몇십 개에서 멈추고, 어절은 "단어 × 조사"의 조합만큼 계속 늘어납니다.

분석기도 틀립니다. Okt는 "영화가 정말 재밌다"에서 부사 "정말"에 명사(Noun) 품사를 붙였습니다. 형태소 분석기도 규칙과 통계로 만든 도구라서 오류가 있고, 분석기마다 결과가 다릅니다. 그래서 전처리 결과는 눈으로 몇 개 확인해 보는 습관이 필요합니다.



그림 2 실습 1의 결과. 위 두 줄은 같은 영어 문장을 두 도구로, 아래 두 줄은 같은 한국어 문장을 띄어쓰기와 형태소로 자른 것

비유

레고 분해하기

완성된 레고 성을 다른 모양으로 다시 조립하려면 먼저 블록으로 분해해야 합니다. 큰 덩어리째 떼어 내면 다시 쓰기 어렵고, 너무 잘게 부수면 의미가 없습니다. 한국어는 블록이 접착제(조사)로 붙어 있어서, 접착 부분까지 떼어 내는 분석기가 필요합니다.

레고 성	= = 문장
블록	= = 토큰
접착제로 붙은 블록	= = 조사가 붙은 어절
블록의 종류표	= = 품사

실습 1

NLTK와 KoNLPy로 토큰화하기

```
# 코랩에서는 먼저:  !pip install konlpy
# 그리고 nltk.download('punkt_tab'), nltk.download('averaged_perceptron_tagger_eng')
from nltk.tokenize import word_tokenize, WordPunctTokenizer, sent_tokenize
from nltk.tag import pos_tag
from konlpy.tag import Okt

# 1) 단어 토큰화: 도구마다 Don't를 다르게 자른다
text = "Don't be fooled by the dark sounding name, Mr. Jone's Orphanage."
print("word_tokenize :", word_tokenize(text)[:9])
print("WordPunct     :", WordPunctTokenizer().tokenize(text)[:9])

# 2) 문장 토큰화: Ph.D.의 마침표에서는 자르지 않는다
text2 = "I am looking for Ph.D. students. and you are a Ph.D. student."
print("문장 토큰화   :", sent_tokenize(text2))

# 3) 품사 태깅 (PRP 대명사, VBP 동사, VBG 현재분사, IN 전치사)
print("품사 태깅     :", pos_tag(word_tokenize(text2))[:4])

# 4) 한국어: 띄어쓰기와 형태소
okt = Okt()                                    # 한국어 형태소 분석기
ko = "열심히 코딩한 당신, 연휴에는 여행을 가봐요"
print("띄어쓰기      :", ko.split())
print("형태소        :", okt.morphs(ko))
print("명사만        :", okt.nouns(ko))
```

실행 결과

```
word_tokenize : ['Do', "n't", 'be', 'fooled', 'by', 'the', 'dark', 'sounding', 'name']
WordPunct     : ['Don', "'", 't', 'be', 'fooled', 'by', 'the', 'dark', 'sounding']
문장 토큰화   : ['I am looking for Ph.D. students.', 'and you are a Ph.D. student.']
품사 태깅     : [('I', 'PRP'), ('am', 'VBP'), ('looking', 'VBG'), ('for', 'IN')]
띄어쓰기      : ['열심히', '코딩한', '당신,', '연휴에는', '여행을', '가봐요']
형태소        : ['열심히', '코딩', '한', '당신', ',', '연휴', '에는', '여행', '을', '가봐요']
명사만        : ['코딩', '당신', '연휴', '여행']
```

직접 해 보기 "아버지가방에들어가신다"처럼 띄어쓰기가 없는 문장을 `okt.morphs()`에 넣어 보세요. 사람은 어떻게 읽고, 분석기는 어떻게 자르나요?

실습 2

조사가 달라도 같은 단어로 모으기

```
# 코랩에서는 먼저:  !pip install konlpy
from konlpy.tag import Okt

# 모두 "영화"에 대한 리뷰지만, 조사가 달라서 어절의 모양이 다르다
reviews = ["영화가 정말 재밌다", "영화를 또 보고 싶다",
           "영화는 지루했다", "영화의 음악이 좋다"]
okt = Okt()

# 1) 띄어쓰기(어절)로 자르기
space = [w for r in reviews for w in r.split()]
movie_like = sorted({w for w in space if w.startswith("영화")})
print("띄어쓰기 토큰:", space)
print("  '영화'가 들어간 서로 다른 토큰:", movie_like, f"→ {len(movie_like)}종류")

# 2) 형태소로 자르기
morph = [w for r in reviews for w in okt.morphs(r)]
print("형태소 토큰  :", morph)
```

```
print("  토큰 '영화'가 나온 횟수:", morph.count("영화"))

# 3) 단어 집합 크기 비교 (서로 다른 토큰의 수)
print("서로 다른 토큰 수: 띄어쓰기", len(set(space)), "개,",
      "형태소", len(set(morph)), "개")

# 4) 품사 태깅: 토큰마다 품사를 붙인다
print("품사 태깅:", okt.pos(reviews[0]))
```

실행 결과

```
띄어쓰기 토큰: ['영화가', '정말', '재밌다', '영화를', '또', '보고', '싶다', '영화는', '지루했다', '영화의',
'음악이', '좋다']
  '영화'가 들어간 서로 다른 토큰: ['영화가', '영화는', '영화를', '영화의'] → 4종류
형태소 토큰  : ['영화', '가', '정말', '재밌다', '영화', '를', '또', '보고', '싶다', '영화', '는', '지루했다',
'영화', '의', '음악', '이', '좋다']
  토큰 '영화'가 나온 횟수: 4
서로 다른 토큰 수: 띄어쓰기 12 개, 형태소 14 개
품사 태깅: [('영화', 'Noun'), ('가', 'Josa'), ('정말', 'Noun'), ('재밌다', 'Adjective')]
```

직접 해 보기 `okt.morphs(r)` 를 `okt.nouns(r)` (명사만 뽑기)로 바꾸면 서로 다른 토큰 수가 몇 개가 되나요? 잃어버린 정보는 무엇인가요?

확인 문제

09-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 한국어는 띄어쓰기만으로 충분히 좋은 토큰화를 할 수 있다. (O / X) 이름 _____

2 구두점은 항상 지워도 문제가 없다. (O / X) 이름 _____

3 뜻을 가진 가장 작은 말의 단위는? 이름 _____

① 문장 ② 어절 ③ 형태소

4 실습에서 "Don't"를 Do와 n't로 자른 도구는? 이름 _____

① word_tokenize ② WordPunctTokenizer ③ split()

5 각 토큰이 명사, 조사 등 무엇인지 표시하는 것을 _____ 태깅이라고 한다. 이름 _____

6 실습 2에서 띄어쓰기로 자르면 "영화"가 들어간 토큰이 _____종류였다. 이름 _____

7 교착어를 정의하고, "영화가 / 영화를" 예시로 한국어 토큰화가 어려운 이유를 설명해 보세요. 이름 _____

원문: 09-01 토큰화 — https://wikidocs.net/217237 (한국어 예문을 영화 리뷰로 바꿔 설명했습니다)

09-02 텍스트 데이터의 정제와 정규화

핵심 한 줄 **정제는 노이즈를 지우는 일이고, 정규화는 표기가 다른 같은 단어를 하나로 맞추는 일이다.**

왜 배우나 컴퓨터에게 "USA", "US", "U.S.A"는 완전히 다른 세 단어입니다. 같은 뜻인데 따로 세면 데이터가 흩어지고, 쓸데없는 기호나 너무 드문 단어는 단어 집합만 키웁니다. 토큰화 전후로 데이터를 다듬어야 모델이 중요한 패턴에 집중할 수 있습니다.

1) 정제와 정규화의 정의

정의

- 정제(cleaning): 말뭉치에서 **노이즈 데이터**를 제거하는 것. 노이즈는 분석 목적에 도움이 되지 않는 글자나 단어입니다(특수 문자, HTML 태그, 의미 없는 단어).
- 정규화(normalization): **표현 방법이 다른 단어들을 통합**해 같은 단어로 만드는 것
- 정제는 토큰화 전에도(토큰화를 방해하는 것 제거), 토큰화 후에도(남은 노이즈 제거) 합니다. 완벽하게 하기는 어려워서 "이 정도면 됐다"는 합의점을 정합니다.

방법	구분	무엇을 하나	주의할 점
규칙으로 표기 통합	정규화	같은 뜻의 다른 표기를 하나로 (USA, US → us)	규칙을 직접 정해야 함
대소문자 통합	정규화	대부분 소문자로 바꿈. 단어 종류가 줄어듦	US(미국)와 us(우리)처럼 대문자가 뜻을 가르는 경우, 회사 이름·사람 이름
빈도가 적은 단어 제거	정제	아주 드물게 나온 단어를 버림	드물지만 중요한 단어도 함께 버려질 수 있음
길이가 짧은 단어 제거	정제	영어의 1~2글자 단어를 버림 (a, I, it, at)	한국어에는 맞지 않음

왜 빈도가 적은 단어를 지울까? 메일 10만 통에서 5번밖에 안 나온 단어는 스팸인지 아닌지를 가리는 규칙을 배우기에 예시가 너무 적습니다. 배우지도 못하면서 단어 집합의 자리만 차지합니다.

왜 짧은 단어 제거는 한국어에 맞지 않을까? 영어 단어의 평균 길이는 6~7글자, 한국어는 2~3글자입니다. 한국어는 한자어가 많아 한 글자에 뜻이 담깁니다. "용(龍)" 한 글자가 영어로는 dragon 여섯 글자입니다. 한국어에서 2글자 이하를 지우면 대부분의 단어가 사라집니다.

예시 적용 리뷰 네 개 "The USA is GREAT", "I love the US", "great food in the U.S.A", "Usa is a big country"에 적용합니다 (실습 3).

- **정규화 전**: USA, US, U.S.A, Usa가 모두 다른 단어이고, GREAT와 great, The와 the도 다릅니다. 단어 종류는 16개입니다.
- **소문자로 통합하고 usa, u.s.a를 us로 통합한 뒤**: 단어 종류가 11개로 줄고, us가 4번으로 가장 많이 나온 단어가 됩니다. 정규화 전에는 네 표기가 1번씩 흩어져 있었습니다.
- **빈도 2 이상만 남기면**: us, the, is, great만 남습니다. 그런데 "love", "food"처럼 리뷰의 내용을 담은 단어도 1번뿐이라 함께 사라집니다. 데이터가 작을 때는 이 방법이 위험하다는 것을 보여 줍니다.

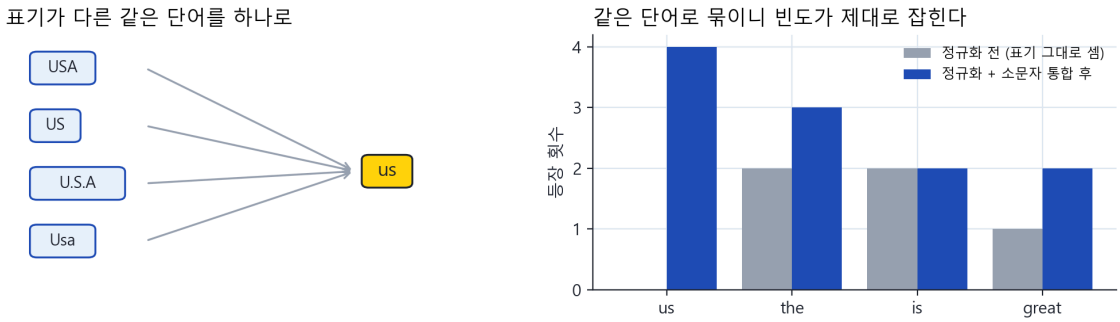


그림 3 왼쪽: 네 가지 표기를 us 하나로 통합. 오른쪽: 통합 전후의 등장 횟수. 흩어져 있던 횟수가 모인다

비유

장보기 영수증 정리

가계부를 정리할 때 "우유 1L", "서울우유", "우유(대)"를 모두 "우유"로 맞춰야 한 달에 우유를 얼마나 샀는지 알 수 있습니다(정규화). 그리고 영수증 맨 아래의 광고 문구나 바코드 숫자는 지웁니다(정제).

품목 이름 맞추기 = = 정규화
광고 문구 지우기 = = 정제
품목별 합계 = = 단어 빈도

실습 3

정규화, 빈도 필터, 짧은 단어 제거

```python
import re
from collections import Counter

reviews = ["The USA is GREAT", "I love the US",
           "great food in the U.S.A", "Usa is a big country"]

# 아무것도 하지 않고 띄어쓰기로만 자른 경우
raw = [w for r in reviews for w in r.split()]

# 1) 대소문자 통합(모두 소문자) + 2) 규칙으로 표기 통합
same_word = {"usa": "us", "u.s.a": "us"}
tokens = [same_word.get(w, w) for r in reviews for w in r.lower().split()]
print("단어 종류: 정규화 전", len(set(raw)), "개 → 후", len(set(tokens)), "개")
print("정규화 후  :", tokens)

# 3) 등장 빈도가 1번뿐인 단어 제거
counts = Counter(tokens)
print("빈도       :", dict(counts.most_common(4)))
print("빈도 2 이상:", [w for w in tokens if counts[w] ⩾ 2])

# 4) 길이가 1~2인 짧은 단어 제거 (정규 표현식, 09-04)
text = "I was wondering if anyone out there could enlighten me on this car."
shortword = re.compile(r"\W*\b\w{1,2}\b")
print("짧은 단어 제거:", shortword.sub("", text))
```

```
실행 결과
단어 종류: 정규화 전 16 개 → 후 11 개
정규화 후  : ['the', 'us', 'is', 'great', 'i', 'love', 'the', 'us', 'great', 'food', 'in', 'the', 'us',
'us', 'is', 'a', 'big', 'country']
빈도       : {'us': 4, 'the': 3, 'is': 2, 'great': 2}
빈도 2 이상: ['the', 'us', 'is', 'great', 'the', 'us', 'great', 'the', 'us', 'us', 'is']
짧은 단어 제거:  was wondering anyone out there could enlighten this car.
```

직접 해 보기 짧은 단어 제거 결과에서 "I", "if", "me", "on"이 사라졌습니다. 정규식의 `{1,2}` 를 `{1,3}` 으로 바꾸면 무엇이 더 사라지나요? 그래도 괜찮은가요?

09-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 USA와 US를 같은 단어로 맞추는 것은 정규화다. (O / X) 이름 _____

2 한국어에서도 길이가 1~2인 단어는 대부분 지워도 된다. (O / X) 이름 _____

3 대소문자를 통합할 때 주의해야 하는 경우는? 이름 _____

① 대문자가 뜻을 구분할 때 (US와 us) ② 문장이 짧을 때 ③ 숫자가 있을 때

4 노이즈 데이터를 지우는 작업은? 이름 _____

① 정제 ② 패딩 ③ 토큰화

5 실습에서 정규화 뒤 단어 종류는 16개에서 _____개로 줄었다. 이름 _____

6 실습에서 정규화 뒤 가장 많이 나온 단어는 _____이다. 이름 _____

7 정제와 정규화를 각각 정의하고, 실습의 리뷰 네 개에서 한 일을 예로 들어 설명해 보세요. 이름 _____

원문: 09-02 텍스트 데이터의 정제와 정규화 — https://wikidocs.net/217238

09-03 불용어

핵심 한 줄 **불용어는 자주 나오지만 글의 내용을 구분하는 데 거의 도움이 안 되는 단어이고, 목록을 만들어 토큰에서 걸러 낸다.**

왜 배우나 영어의 the, is, a나 한국어의 조사 "를", "는"은 거의 모든 문장에 나옵니다. 긍정 리뷰에도 부정 리뷰에도 똑같이 나오므로 둘을 가르는 단서가 되지 못하면서 토큰의 수만 늘립니다. 09-02의 정제 가운데 가장 흔히 쓰는 방법이라 따로 다룹니다.

1) 불용어의 정의와 제거

정의

- 불용어(stopword): 말뭉치에 자주 등장하지만 분석에 큰 도움이 되지 않는 단어. 조사, 접속사, 관사, 대명사 등이 많습니다.
- **제거 방법**: 불용어 **목록**을 준비하고, 토큰마다 목록에 있는지 확인해 있으면 뺍니다.
- **영어**: NLTK 패키지에 미리 정의된 목록이 있습니다.
- **한국어**: 정해진 목록이 없습니다. 형태소로 자른 뒤 조사와 접속사를 빼거나, 분석 목적에 맞게 목록을 직접 만들어 파일로 관리합니다.

무엇이 불용어인지는 하려는 일에 따라 달라집니다.

- 부정어 not, no, "안", "못"은 흔한 단어지만 뜻을 뒤집습니다. 긍정/부정을 가르는 일에서는 지우면 안 됩니다.
- 다음 단어를 맞히는 언어 모델(10장)이나 번역은 자연스러운 문장을 만들어야 하므로 조사와 관사도 필요합니다. 이런 일에서는 불용어를 지우지 않습니다.

예시 적용 NLTK의 영어 불용어는 198개입니다(버전에 따라 달라서 교재에는 179개로 나옵니다). "Family is not an important thing. It's everything."에서 불용어를 빼면 "Family important thing. 's everything."이 됩니다(실습 4). is, an, It과 함께 **not도 지워져서** "가족은 중요한 것이 아니다"가 "가족, 중요한 것"으로 바뀌었습니다. 뜻이 반대가 됐습니다. 영화 리뷰라면 "not good"이 "good"이 되는 셈입니다.

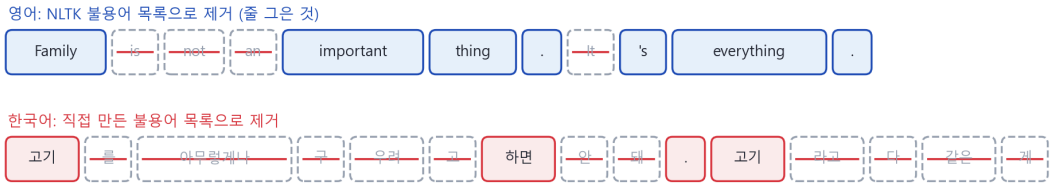


그림 4 실습 4의 결과. 줄 그은 토큰이 불용어로 제거된 것이다. 영어의 not, 한국어의 "안"도 함께 지워졌다

한국어 예문 "고기를 아무렇게나 구우려고 하면 안 돼. 고기라고 다 같은 게 아니거든."은 직접 만든 목록으로 걸렀습니다. 남은 것은 "고기, 하면, 고기, 아니거든"입니다. "안"과 "돼"를 목록에 넣었기 때문에 "하면 안 돼"의 뜻도 사라졌습니다. 목록을 만드는 사람이 무엇을 남길지 판단해야 합니다.

비유

회의록 요약하기

회의 녹음을 받아 적으면 "음", "그러니까", "어쨌든" 같은 말이 많이 나옵니다. 요약할 때는 이런 말을 빼고 "예산", "마감일" 같은 핵심어만 남깁니다. 하지만 "안 됩니다"의 "안"처럼 결론을 뒤집는 말까지 빼면 회의 결과가 반대로 적힙니다.

음, 그러니까 = = 불용어
예산, 마감일 = = 남길 내용어
안 됩니다의 "안" = = 지우면 안 되는 부정어

실습 4

NLTK 불용어와 한국어 사용자 정의 불용어

```
# 코랩에서는 먼저:  !pip install konlpy   그리고  nltk.download('stopwords')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from konlpy.tag import Okt

en_stop = set(stopwords.words("english"))
print("영어 불용어:", len(en_stop), "개, 예:", sorted(en_stop)[:8])
print("'not'도 불용어인가?", "not" in en_stop)

example = "Family is not an important thing. It's everything."
tokens = word_tokenize(example)
print("제거 전:", tokens)
print("제거 후:", [w for w in tokens if w.lower() not in en_stop])

# 한국어는 정해진 목록이 없어서 직접 정의한다
okt = Okt()
ko = "고기를 아무렇게나 구우려고 하면 안 돼. 고기라고 다 같은 게 아니거든."
ko_stop = {"를", "아무렇게나", "구", "우려", "고", "안", "돼",
           "같은", "게", "라고", "다"}
ko_tokens = okt.morphs(ko)
print("한국어 제거 전:", ko_tokens)
print("한국어 제거 후:", [w for w in ko_tokens if w not in ko_stop])
```

```
실행 결과
영어 불용어: 198 개, 예: ['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all']
'not'도 불용어인가? True
제거 전: ['Family', 'is', 'not', 'an', 'important', 'thing', '.', 'It', "'s", 'everything', '.']
제거 후: ['Family', 'important', 'thing', '.', "'s", 'everything', '.']
한국어 제거 전: ['고기', '를', '아무렇게나', '구', '우려', '고', '하면', '안', '돼', '.', '고기', '라고',
'다', '같은', '게', '아니거든', '.']
한국어 제거 후: ['고기', '하면', '.', '고기', '아니거든', '.']
```

직접 해 보기 영어 불용어 목록에서 "not"을 빼고(`en_stop.discard("not")`) 다시 실행해 보세요. 결과 문장의 뜻이 어떻게 달라지나요?

09-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 불용어는 자주 나오지만 내용 구분에 큰 도움이 안 되는 단어다. (O / X) 이름 _____

2 NLTK에는 한국어 불용어 목록이 기본으로 들어 있다. (O / X) 이름 _____

3 긍정/부정 분류에서 불용어로 지우면 위험한 단어는? 이름 _____

① the ② not ③ a

4 불용어를 지우지 않는 편이 좋은 일은? 이름 _____

① 다음 단어를 맞히는 언어 모델 ② 뉴스 기사의 주제 분류 ③ 핵심어 뽑기

5 실습에서 NLTK 영어 불용어는 _____개였다. 이름 _____

6 실습의 한국어 예문에서 불용어를 지운 뒤 남은 명사는 _____(두 번)이다. 이름 _____

7 불용어를 정의하고, "not good" 예시로 불용어 제거의 주의점을 설명해 보세요. 이름 _____

원문: 09-03 불용어 — https://wikidocs.net/217239

09-04 정규 표현식

핵심 한 줄

정규 표현식은 "숫자 3개-4개-4개"처럼 글자의 패턴을 기호로 적어, 그 패턴에 맞는 부분을 찾고, 자르고, 바꾸는 도구다.

왜 배우나

09-02의 "짧은 단어 지우기", 09-05의 "한글만 남기기"는 모두 정규 표현식 한 줄로 합니다. 전화번호, 이메일, 해시태그처럼 모양이 정해진 정보를 뽑거나 특수 문자를 한 번에 지울 때 반복문 수십 줄을 대신해 줍니다. 정제에서 가장 자주 쓰는 도구입니다.

1) 정규 표현식 문법

정의

정규 표현식(regular expression)은 문자열의 **패턴**을 기호로 나타내는 표기법입니다. 기호는 세 종류로 나눠 보면 쉽습니다.

종류	기호	뜻	예 (패턴 → 맞는 글자)
무슨 글자인가	`.`	아무 글자 하나 (줄바꿈 제외)	`a.c` → abc, a1c
	`[abc]` `[a-c]`	괄호 안 글자 중 하나. `-` 는 범위	`[a-c]` → a, b, c 중 하나
	`[^abc]`	괄호 안 글자가 **아닌** 글자 하나	`[^0-9]` → 숫자가 아닌 글자
몇 번 나오는가 (바로 앞 글자에 붙음)	`?`	0번 또는 1번	`ab?c` → ac, abc
	`*`	0번 이상	`ab*c` → ac, abc, abbc
	`+`	1번 이상	`ab+c` → abc, abbc (ac는 안 됨)
	`{n}` `{n,m}` `{n,}`	정확히 n번 / n~m번 / n번 이상	`ab{2}c` → abbc
어디에 있는가	`^`	문자열의 시작	`^ab` → ab로 시작하는 글
	`$`	문자열의 끝	`ab$` → ab로 끝나는 글

자주 쓰는 글자 묶음에는 줄임 기호가 있습니다. 대문자는 "그것이 아닌 것"입니다.

기호	뜻	같은 뜻의 표기
`\d` / `\D`	숫자 / 숫자가 아닌 것	`[0-9]` / `[^0-9]`
`\w` / `\W`	글자·숫자·밑줄 / 그것이 아닌 것	`[a-zA-Z0-9_]` (한글도 포함) / 그 반대
`\s` / `\S`	공백(띄어쓰기, 탭, 줄바꿈) / 공백이 아닌 것	`[ \t\n\r\f\v]` / 그 반대

`^` 는 자리에 따라 뜻이 다릅니다. 패턴 맨 앞에 있으면 "시작", 대괄호 `[ ]` 안의 맨 앞에 있으면 "아닌 것"입니다. 기호 자체를 찾고 싶을 때는 앞에 역슬래시를 붙입니다(`\.` 은 마침표, `\+` 는 더하기). 파이썬에서는 패턴을 `r"..."` 로 쓰면 역슬래시가 그대로 전달됩니다.

예시 적용

패턴 뜯어 읽기 1. 전화번호 패턴 `\d{3}-\d{4}-\d{4}` 는 "숫자 3개, 하이픈, 숫자 4개, 하이픈, 숫자 4개"입니다. 회원 정보가 적힌 긴 글에서 이 모양인 곳만 찾아냅니다(실습 5).

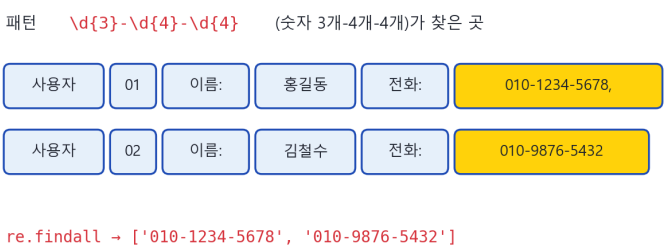
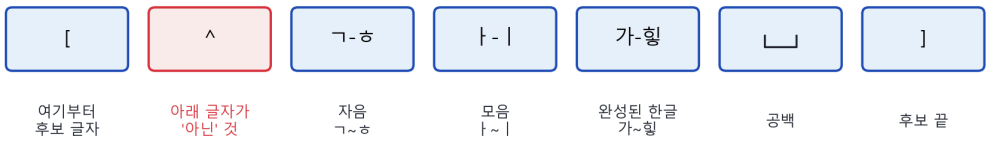


그림 5 패턴에 맞는 부분(노랑)만 찾아낸다. "01", "02" 같은 숫자는 모양이 달라서 걸리지 않는다

패턴 뜯어 읽기 2. 리뷰를 정제할 때 쓰는 `[^ㄱ-ㅎㅏ-ㅣ가-힣 ]`는 길어 보이지만 대괄호 하나입니다.



여기부터 후보 글자 | 아래 글자가 '아닌' 것 | 자음 ㄱ~ㅎ | 모음 ㅏ~ㅣ | 완성된 한글 가~힣 | 공백 | 후보 끝

전체 뜻: 한글(자음, 모음, 완성형)과 공백이 '아닌' 글자 하나 → re.sub(패턴, "", 글)로 지우면 한글과 공백만 남는다

그림 6 "한글과 공백이 아닌 글자 하나"라는 패턴. 이것을 빈 문자열로 바꾸면 한글과 공백만 남는다

"Hi!! 오늘 날씨 너무 좋아요 :) #행복 #주말"에 적용하면 영어, 느낌표, 기호가 지워지고 "오늘 날씨 너무 좋아요 행복 주말"이 남습니다. 자음만 쓴 "ㅋㅋ", 모음만 쓴 "ㅠㅠ"는 `ㄱ-ㅎ`, `ㅏ-ㅣ` 에 들어 있으므로 지워지지 않습니다.

2) 파이썬 re 모듈의 함수

정의

패턴은 "무엇을 찾을지"이고, 함수는 "찾아서 무엇을 할지"입니다. 파이썬은 `re` 모듈로 정규 표현식을 씁니다.

함수	하는 일	돌려주는 것
`re.search(패턴, 글)`	글 **전체**에서 패턴에 맞는 첫 부분을 찾음	Match 객체 (없으면 None)
`re.match(패턴, 글)`	글의 **맨 앞**이 패턴에 맞는지 확인	Match 객체 (없으면 None)
`re.findall(패턴, 글)`	패턴에 맞는 부분을 모두 찾음	문자열 리스트
`re.split(패턴, 글)`	패턴을 기준으로 글을 자름	문자열 리스트
`re.sub(패턴, 바꿀 글, 글)`	패턴에 맞는 부분을 다른 글로 바꿈	바뀐 문자열
`re.compile(패턴)`	패턴을 미리 만들어 둠. 같은 패턴을 여러 번 쓸 때 빠름	패턴 객체 (`.sub()` , `.findall()` 등을 가짐)

전처리에서는 지우기(`sub` 로 빈 문자열로 바꾸기)와 뽑기(`findall`)를 가장 많이 씁니다. Match 객체의 `span=(3, 6)` 은 "3번 자리부터 6번 자리 앞까지가 맞았다"는 뜻입니다.

예시 적용

쇼핑몰 리뷰 "배송 3일, 가격 15000원, 별점 5점!! 재구매 100%"에 같은 패턴 `\d+` (숫자가 1개 이상 이어진 것)를 함수만 바꿔 적용합니다(실습 6).

함수	결과
`re.findall(r"\d+", 리뷰)`	`['3', '15000', '5', '100']`
`re.sub(r"\d+", "N", 리뷰)`	`배송 N일, 가격 N원, 별점 N점!! 재구매 N%`

`search` 와 `match` 의 차이도 확인합니다. "kkkabc"에서 패턴 `ab.` 을 찾으면 `search` 는 3번 자리의 "abc"를 찾지만, `match` 는 맨 앞 글자가 k라서 None을 돌려줍니다.

09-01의 토큰화도 정규 표현식으로 할 수 있습니다. `re.findall(r"\w+", 글)` 은 글자와 숫자가 이어진 덩어리만 꺼내므로, "Don't"는 `Don` 과 `t` 로, "Mr."는 `Mr` 로 나옵니다. 구두점을 모두 버리는 가장 단순한 토큰화입니다.

비유

택배 송장의 빈칸 양식

"□□□-□□□□-□□□□"라는 양식을 보면 우리는 바로 전화번호 칸인 줄 압니다. 정규 표현식은 이런 빈칸 양식을 기호로 적은 것이고, 컴퓨터는 긴 글에서 그 양식에 딱 맞는 부분을 찾아냅니다.

- □ 한 칸 (숫자) == `\d`
- □□□ (세 칸) == `\d{3}`
- 양식에 맞는 곳 모두 찾기 == `re.findall()`
- 찾은 곳을 지우개로 지우기 == `re.sub(패턴, "", 글)`

실습 5

기호별 동작과 실전 패턴

```
import re

print(re.search("a.c", "kkkabc"))      # .  : 아무 글자 하나
print(re.search("ab?c", "ac"))         # ?  : 앞 글자가 있어도 없어도 됨
print(re.search("ab*c", "abbbc"))      # *  : 앞 글자가 0번 이상
print(re.search("ab+c", "ac"))         # +  : 앞 글자가 1번 이상 → None
print(re.search("^ab", "zab"))         # ^  : ab로 시작해야 함 → None
print(re.search("ab{2}c", "abbc"))     # {2}: 앞 글자가 정확히 2번
print(re.search("[a-c]", "zzzb"))      # [] : 괄호 안 글자 중 하나

text = ("사용자 01 이름: 홍길동 전화: 010-1234-5678, "
        "사용자 02 이름: 김철수 전화: 010-9876-5432")
print("전화번호 찾기:", re.findall(r"\d{3}-\d{4}-\d{4}", text))
print("숫자 지우기  :", re.sub(r"[0-9-]+", "", text)[:24])

sentence = "Hi!! 오늘 날씨 너무 좋아요 :) #행복 #주말"
korean = re.sub(r"[^ㄱ-ㅎㅏ-ㅣ가-힣 ]", "", sentence)
print("한글만 남기기:", korean.split())
print("해시태그만   :", re.findall(r"#\w+", sentence))
```

실행 결과

```
<re.Match object; span=(3, 6), match='abc'>
<re.Match object; span=(0, 2), match='ac'>
<re.Match object; span=(0, 5), match='abbbc'>
None
None
<re.Match object; span=(0, 4), match='abbc'>
<re.Match object; span=(3, 4), match='b'>
전화번호 찾기: ['010-1234-5678', '010-9876-5432']
숫자 지우기  : 사용자  이름: 홍길동 전화: , 사용자
한글만 남기기: ['오늘', '날씨', '너무', '좋아요', '행복', '주말']
해시태그만   : ['#행복', '#주말']
```

직접 해 보기 이메일 주소를 찾는 패턴을 만들어 보세요. 힌트: `[\w.]+@\w+\.\w+` . "문의: help@study.com 또는 me.kim@mail.net"에서 둘 다 찾아지나요?

실습 6 re 모듈 함수: search, match, split, findall, sub, compile

```python
import re

# 1) match는 문자열의 "맨 앞"부터 맞아야 하고, search는 어디든 찾는다
print("search:", re.search("ab.", "kkkabc"))
print("match :", re.match("ab.", "kkkabc"), " ← 맨 앞이 k라서 실패")
print("match :", re.match("ab.", "abckkk"))

# 2) split: 패턴을 기준으로 자른다
print("split 공백 :", re.split(" ", "사과 딸기 수박"))
print("split +    :", re.split(r"\+", "사과+딸기+수박"))

# 3) findall: 맞는 것을 모두 리스트로 / sub: 맞는 것을 바꾼다
review = "배송 3일, 가격 15000원, 별점 5점!! 재구매 100%"
print("findall 숫자:", re.findall(r"\d+", review))
print("sub 숫자→N  :", re.sub(r"\d+", "N", review))

# 4) compile: 같은 패턴을 여러 번 쓸 때 미리 만들어 둔다
only_korean = re.compile(r"[^ㄱ-ㅎㅏ-ㅣ가-힣 ]")       # 한글과 공백이 아닌 글자
for text in ["이 영화 정말 재밌어요!! ㅋㅋ", "Best movie... 최고 10/10"]:
    print("한글만:", only_korean.sub("", text))

# 5) 정규 표현식으로 토큰화: 글자·숫자가 이어진 덩어리만 꺼낸다
text = "Don't be fooled, Mr. Jone's Orphanage is cheery."
print("토큰화 \\w+ :", re.findall(r"\w+", text))
```

실행 결과

```
search: <re.Match object; span=(3, 6), match='abc'>
match : None  ← 맨 앞이 k라서 실패
match : <re.Match object; span=(0, 3), match='abc'>
split 공백 : ['사과', '딸기', '수박']
split +    : ['사과', '딸기', '수박']
findall 숫자: ['3', '15000', '5', '100']
sub 숫자→N  : 배송 N일, 가격 N원, 별점 N점!! 재구매 N%
한글만: 이 영화 정말 재밌어요 ㅋㅋ
한글만:  최고
토큰화 \w+ : ['Don', 't', 'be', 'fooled', 'Mr', 'Jone', 's', 'Orphanage', 'is', 'cheery']
```

직접 해 보기 두 번째 문장 "Best movie... 최고 10/10"은 한글만 남기니 "최고"만 남았습니다. 영어 리뷰도 살리려면 패턴에 무엇을 추가해야 할까요? (힌트: `a-zA-Z`)

확인 문제

09-04 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 정규 표현식에서 `.` 은 아무 글자 하나를 뜻한다. (O / X) 이름 _____

2 `re.match` 는 글의 어느 위치에서든 패턴을 찾아 준다. (O / X) 이름 _____

3 패턴에 맞는 부분을 모두 리스트로 돌려주는 함수는? 이름 _____

① re.findall ② re.sub ③ re.split

4 `[^ㄱ-ㅎㅏ-ㅣ가-힣 ]` 에 맞는 글자를 빈 문자열로 바꾸면? 이름 _____

① 한글만 지워진다 ② 한글과 공백만 남는다 ③ 아무 변화 없다

5 숫자 한 글자를 뜻하는 줄임 기호는 _____이다. 이름 _____

6 `ab+c` 에서 `+` 는 바로 앞 글자가 _____번 이상 나온다는 뜻이다. 이름 _____

7 패턴 `\d{3}-\d{4}-\d{4}` 를 기호 하나씩 뜯어 읽어 옆 사람에게 설명해 보세요. 이름 _____

원문: 09-04 정규 표현식 — https://wikidocs.net/217240

09-05 딥 러닝을 위한 자연어 처리 전처리 실습

핵심 한 줄 **토큰화 → 단어 집합 → 정수 인코딩 → 패딩을 거치면, 길이가 제각각인 문장들이 (문장 수, 최대 길이) 모양의 정수 텐서가 된다.**

왜 배우나 09-01~09-04의 도구를 한 줄로 이어 붙이는 소챕터입니다. 신경망은 숫자만, 그리고 **같은 크기로 묶인 텐서**만 한꺼번에 처리할 수 있습니다(07-01의 배치). 여기서 만드는 정수 텐서는 12장에서 `nn.Embedding`을 거쳐 벡터가 되고, 13장의 RNN 텍스트 분류기에 그대로 들어갑니다.

1) 전처리의 전체 흐름

정의

$$\text{문장} \xrightarrow{\text{정제}} \text{깨끗한 문장} \xrightarrow{\text{토큰화}} \text{토큰 리스트} \xrightarrow{\text{정수 인코딩}} \text{정수 리스트} \xrightarrow{\text{패딩}} \text{텐서 } (N,\ L)$$

N은 문장 수, L은 맞춘 길이입니다. 정수 인코딩에는 미리 만든 단어 집합이 필요합니다.

토큰화는 어떤 단위로 자를지부터 정합니다. 교재는 다섯 가지를 보여 줍니다.

단위	방법	"영화가 재밌다"를 자르면	특징
띄어쓰기	`split()`	영화가 / 재밌다	가장 단순. 한국어에는 부족 (09-01)
단어 (영어)	spaCy, NLTK	(영어용)	구두점과 접어를 규칙에 따라 분리
형태소	KoNLPy의 Okt, Mecab	영화 / 가 / 재밌다	한국어에서 기본으로 쓰는 방법
문자	`list()`	영 / 화 / 가 / 재 / 밌 / 다	단어 집합이 작아짐. 07-03의 Char RNN

예시 적용 영화 리뷰 6개로 따라갑니다. 첫 리뷰 "이 영화 정말 재밌어요!! 배우 연기가 최고"는 이렇게 바뀝니다(실습 7).

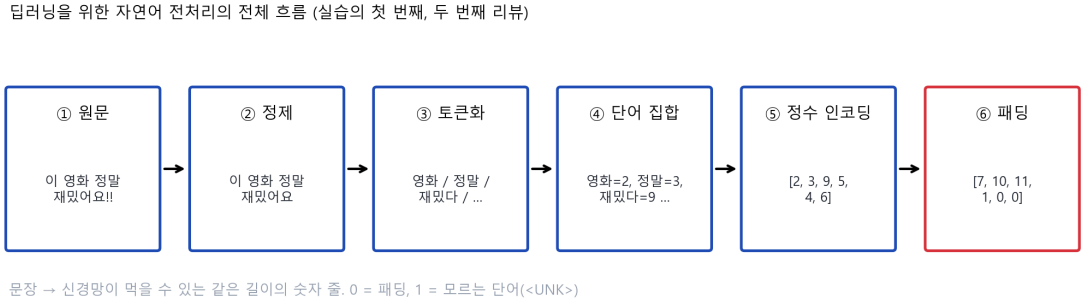


그림 7 리뷰 한 줄이 여섯 단계를 거쳐 숫자 줄이 된다. 마지막 칸은 길이가 짧은 둘째 리뷰가 패딩된 모습

정제는 09-04의 `[^ㄱ-ㅎㅏ-ㅣ가-힣 ]`로 한글과 공백만 남기고, 토큰화는 Okt로 하되 `stem=True`를 주어 "재밌어요"를 기본형 "재밌다"로 바꿉니다(정규화). 그다음 직접 만든 불용어 목록("이", "가", "도", "는" 등)을 뺍니다.

2) 단어 집합과 정수 인코딩

정의

- 단어 집합(vocabulary): 말뭉치에 나온 서로 다른 토큰의 목록에 번호를 붙인 것. 크기를 V라고 합니다.
- 정수 인코딩: 단어 집합을 보고 토큰을 번호로 바꾸는 것
- <UNK>(unknown): 단어 집합에 없는 단어를 대신하는 특별한 토큰. 이런 단어를 OOV(Out-Of-Vocabulary)라고 합니다.

- <PAD>(padding): 길이를 맞추려고 채워 넣는 특별한 토큰. 아무 뜻도 없습니다.

왜 많이 나온 단어부터 번호를 붙일까? 번호 자체에는 뜻이 없지만, 빈도순으로 붙여 두면 "상위 k개만 쓰자"를 번호만 보고 할 수 있습니다. 드문 단어는 09-02에서 본 것처럼 배우기 어려우므로 버리고 <UNK>로 대신합니다.

왜 단어 집합의 크기를 제한할까? V는 모델의 크기와 직결됩니다. 12장의 임베딩 층은 단어마다 벡터를 하나씩 가지고, 출력층은 V개 중 하나를 고릅니다. V가 크면 모델이 무겁고, 작으면 <UNK>가 많아져 정보를 잃습니다.

예시 적용 리뷰 6개에서 나온 서로 다른 토큰은 20개입니다. 그중 많이 나온 10개만 남기고 <PAD> = 0, <UNK> = 1을 더해 $V = 12$인 단어 집합을 만듭니다. 가장 많이 나온 "영화"가 2번, "정말"이 3번입니다.

첫 리뷰 [영화, 정말, 재밌다, 배우, 연기, 최고]는 [2, 3, 9, 5, 4, 6]이 됩니다. 넷째 리뷰 [시간, 아깝다, 영화, 지루함]은 "영화"만 단어 집합에 있어서 [1, 1, 2, 1]이 됩니다. 전체 토큰 30개 중 10개가 <UNK>입니다. 단어 집합을 너무 작게 잡으면 "시간 아깝다" 같은 부정의 단서가 통째로 사라진다는 것을 보여 줍니다.

3) 패딩

정의

- 패딩(padding): 문장들의 길이를 같게 맞추는 것. 정한 길이 L보다 짧은 문장은 뒤에 <PAD>의 번호(0)를 채우고, 긴 문장은 잘라 냅니다.
- L은 가장 긴 문장의 길이로 하거나, 문장 길이의 분포를 보고 "대부분의 문장이 들어가는 길이"로 정합니다.

왜 길이를 맞출까? 텐서는 직사각형이어야 합니다. 길이가 4인 문장과 6인 문장은 한 텐서로 묶을 수 없어서 문장을 하나씩 따로 계산해야 하고, 그러면 매우 느립니다. 길이를 맞추면 $(N,\ L)$ 텐서 하나로 묶어 한꺼번에 계산할 수 있습니다.

왜 0번일까? 0으로 정해야 하는 것은 아니고 관례입니다. 패딩 자리는 뜻이 없으므로 모델이 무시해야 하는데, 번호를 하나로 정해 두면 "이 번호는 건너뛰어라"라고 알려 주기 쉽습니다(12장 `nn.Embedding`의 `padding_idx=0`).

예시 적용 리뷰 6개의 길이는 6, 4, 6, 4, 6, 4입니다. 가장 긴 6에 맞추면 길이 4인 리뷰 세 개 뒤에 0이 두 개씩 붙어, 모양이 (6, 6)인 텐서가 됩니다.

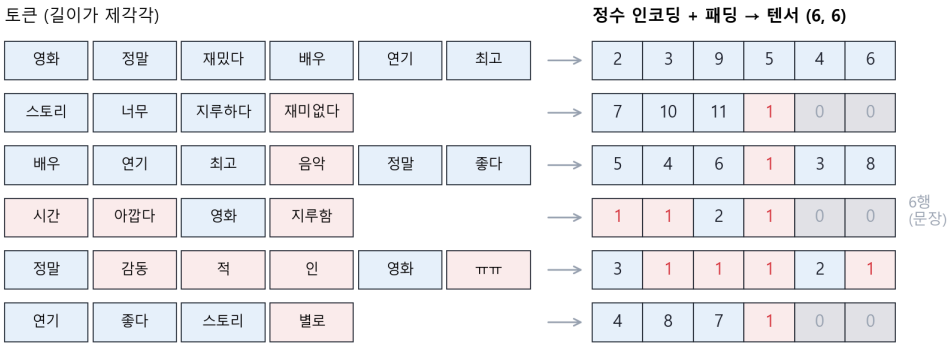


그림 8 실습 7의 결과. 왼쪽의 길이가 다른 토큰 줄이 오른쪽의 6행 × 6열 정수 텐서가 된다. 빨강은 단어 집합에 없어 1이 된 자리, 회색은 패딩

교재는 네이버 영화 리뷰 20만 개 가운데 100개를 뽑아 같은 과정을 진행하며, 단어 집합은 상위 500개, 길이는 가장 긴 리뷰에 맞춥니다. 흐름은 같고 숫자만 큽니다. 코랩에서 교재 코드로 큰 데이터도 돌려 보세요.

비유

도서관 책 정리

새 책이 들어오면 분류 번호를 붙이고(정수 인코딩), 목록에 없는 희귀본은 "기타" 칸에 넣습니다(<UNK>). 책장 칸 크기가 정해져 있어서 얇은 책 묶음은 빈 공간을 받침대로 채워 높이를 맞춥니다(패딩).

분류 번호표	= = 단어 집합
책에 번호 붙이기	= = 정수 인코딩
"기타" 칸	= = <UNK> (1번)
받침대로 높이 맞추기	= = 패딩 (0번)

실습 7

리뷰 6개를 (6, 6) 텐서로 바꾸기

```
# 코랩에서는 먼저:  !pip install konlpy
# 교재는 네이버 영화 리뷰 20만 개(ratings.txt)를 쓴다.
# 여기서는 흐름을 보기 위해 리뷰 6개로 한다.
import re
from collections import Counter

import torch
from konlpy.tag import Okt

reviews = ["이 영화 정말 재밌어요!! 배우 연기가 최고",
           "스토리가 너무 지루하고 재미없어요",
           "배우들 연기 최고, 음악도 정말 좋아요",
           "시간 아까운 영화... 지루함",
           "정말 감동적인 영화예요 ㅠㅠ",
           "연기는 좋은데 스토리가 별로"]

okt = Okt()
stopwords = {"이", "가", "도", "는", "요", "예요", "들", "은", "고"}

# 1) 정제 + 토큰화 + 불용어 제거
tokenized = []
for r in reviews:
    r = re.sub(r"[^ㄱ-ㅎㅏ-ㅣ가-힣 ]", "", r)          # 한글과 공백만 남기기
    tokens = okt.morphs(r, stem=True)                  # 형태소로 자르고 기본형으로
    tokenized.append([w for w in tokens if w not in stopwords])
for sent in tokenized:
    print("토큰:", sent)

# 2) 단어 집합: 많이 나온 순서대로 번호 (0 = 패딩, 1 = 모르는 단어)
counts = Counter(w for sent in tokenized for w in sent)
vocab = {"<PAD>": 0, "<UNK>": 1}
for w, _ in counts.most_common(10):                    # 상위 10개 단어만 보존
    vocab[w] = len(vocab)
print("\n서로 다른 토큰:", len(counts), "개 → 단어 집합:", len(vocab), "개")
print(vocab)

# 3) 정수 인코딩: 단어 → 번호 (단어 집합에 없으면 1번)
encoded = [[vocab.get(w, 1) for w in sent] for sent in tokenized]
print("\n정수 인코딩:", encoded)

# 4) 패딩: 가장 긴 문장에 길이를 맞추고 빈자리는 0으로
max_len = max(len(s) for s in encoded)
padded = [s + [0] * (max_len - len(s)) for s in encoded]
X = torch.tensor(padded)
print("\n텐서 모양:", tuple(X.shape), "= (문장 수, 최대 길이)")
print(X)
print("<UNK>(1)의 수:", int((X == 1).sum()), " <PAD>(0)의 수:", int((X == 0).sum()))
```

실행 결과

```
토큰: ['영화', '정말', '재밌다', '배우', '연기', '최고']
토큰: ['스토리', '너무', '지루하다', '재미없다']
토큰: ['배우', '연기', '최고', '음악', '정말', '좋다']
토큰: ['시간', '아깝다', '영화', '지루함']
토큰: ['정말', '감동', '적', '인', '영화', 'ㅠㅠ']
토큰: ['연기', '좋다', '스토리', '별로']

서로 다른 토큰: 20 개 → 단어 집합: 12 개
{'<PAD>': 0, '<UNK>': 1, '영화': 2, '정말': 3, '연기': 4, '배우': 5, '최고': 6, '스토리': 7, '좋다': 8,
'재밌다': 9, '너무': 10, '지루하다': 11}

정수 인코딩: [[2, 3, 9, 5, 4, 6], [7, 10, 11, 1], [5, 4, 6, 1, 3, 8], [1, 1, 2, 1], [3, 1, 1, 1, 2, 1],
```

```
[4, 8, 7, 1]]

텐서 모양: (6, 6) = (문장 수, 최대 길이)
tensor([[ 2,  3,  9,  5,  4,  6],
        [ 7, 10, 11,  1,  0,  0],
        [ 5,  4,  6,  1,  3,  8],
        [ 1,  1,  2,  1,  0,  0],
        [ 3,  1,  1,  1,  2,  1],
        [ 4,  8,  7,  1,  0,  0]])
<UNK>(1)의 수: 10  <PAD>(0)의 수: 6
```

직접 해 보기 `most_common(10)` 을 `most_common(5)` 로 줄이면 <UNK>(1)이 몇 개로 늘어나나요? 20으로 늘리면 몇 개가 되나요?

확인 문제

09-05 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 패딩은 문장들의 길이를 같게 맞추는 작업이다. (O / X) 이름 _____

2 단어 집합에 없는 단어는 그냥 지워 버린다. (O / X) 이름 _____

3 단어를 번호로 바꾸는 단계는? 이름 _____

① 토큰화 ② 정수 인코딩 ③ 정제

4 문장의 길이를 맞춰야 하는 이유는? 이름 _____

① 텐서 하나로 묶어 한꺼번에 계산하려고 ② 문장을 짧게 요약하려고 ③ 불용어를 지우려고

5 모르는 단어를 대신하는 특별한 토큰은 <_____>이다. 이름 _____

6 실습에서 만든 텐서의 모양은 (_____, 6)이다. 이름 _____

7 단어 집합의 크기를 너무 작게 잡으면 생기는 일을, 실습의 넷째 리뷰 [1, 1, 2, 1]로 설명해 보세요. 이름 _____

원문: 09-05 딥 러닝을 위한 자연어 처리 전처리 실습 — https://wikidocs.net/64517 (교재의 네이버 영화 리뷰 데이터를 리뷰 6개로 줄여 설명했습니다)

오늘의 요약

다시 보기

토큰화 09-01 — 정제, 정규화 09-02 — 불용어 09-03 — 정규 표현식 09-04 — **정수 텐서 09-05**

정의 모음

개념	정의	예시
토큰화	말뭉치를 토큰(단어, 문장, 형태소)으로 나누기	Don't → Do / n't
형태소, 교착어	뜻을 가진 가장 작은 단위 / 조사·어미가 붙는 언어	영화가, 영화를 → 영화 + 조사
정제 / 정규화	노이즈 제거 / 표기가 다른 같은 단어 통합	USA, US, U.S.A → us (16종 → 11종)
불용어	자주 나오지만 내용 구분에 도움이 안 되는 단어	NLTK 198개. not을 지우면 뜻이 반대
정규 표현식	문자열의 패턴을 기호로 적은 표기법	`\d{3}-\d{4}-\d{4}`, `[^ㄱ-ㅎㅏ-ㅣ가-힣 ]`
단어 집합, 정수 인코딩	토큰에 번호를 붙인 목록 / 토큰을 번호로 바꾸기	영화 = 2, 정말 = 3. $V = 12$
<UNK> / <PAD>	단어 집합에 없는 단어 자리 / 길이 맞춤 자리	1번 (30개 중 10개) / 0번
패딩	문장 길이를 L로 맞춰 (N, L) 텐서로 만들기	리뷰 6개 → (6, 6)

다음

다음 자료 예고: 10장 언어 모델

문장을 토큰으로 자를 수 있게 됐으니, 이제 "이 토큰 다음에 어떤 토큰이 올까"를 확률로 계산해 봅니다. 휴대폰 키보드의 다음 단어 추천부터 GPT까지 이어지는 생각입니다.

정답과 해설

09-01 토큰화

1. X — 조사가 붙어 같은 단어가 다른 토큰이 되므로 형태소 분석이 필요합니다.
2. X — Ph.D., $45.55처럼 구두점이 단어의 일부인 경우가 있습니다.
3. ③
4. ①
5. 품사
6. 4 — 영화가, 영화는, 영화를, 영화의
7. 예시 답: 교착어는 단어 뒤에 조사나 어미가 붙어 문법적 역할을 나타내는 언어다. 띄어쓰기로 자르면 "영화가"와 "영화를"이 서로 다른 토큰이 되어 같은 단어인 줄 모른다. 형태소로 잘라 "영화"와 조사를 나눠야 한다.

09-02 정제와 정규화

1. O
2. X — 한국어 단어의 평균 길이가 2~3글자라서 대부분의 단어가 사라집니다.
3. ①
4. ①
5. 11
6. us (4번)
7. 예시 답: 정제는 노이즈 데이터를 지우는 것, 정규화는 표기가 다른 같은 단어를 통합하는 것이다. 실습에서 USA, U.S.A를 us로 바꾸고 소문자로 통일한 것이 정규화, 한 번만 나온 단어와 1~2글자 단어를 지운 것이 정제다.

09-03 불용어

1. O
2. X — 한국어 목록은 직접 만들어야 합니다.
3. ②
4. ① — 자연스러운 문장을 만들려면 조사와 관사도 필요합니다.
5. 198
6. 고기
7. 예시 답: 불용어는 자주 나오지만 분석에 도움이 안 되는 단어다. NLTK 목록에는 not도 들어 있어서 "not good"에서 not을 지우면 "good"만 남아 뜻이 반대가 된다. 하려는 일에 맞게 목록을 고쳐 써야 한다.

09-04 정규 표현식

1. O
2. X — `match`는 맨 앞만 확인합니다. 어디서든 찾는 것은 `search` 입니다.
3. ①
4. ② — `[^…]`는 괄호 안 글자가 "아닌" 것이므로, 한글과 공백이 아닌 것을 지웁니다.
5. \d
6. 1
7. 예시 답: `\d`는 숫자 하나, `{3}`은 그것이 3번. 그러니 숫자 3개, 하이픈, 숫자 4개, 하이픈, 숫자 4개라는 뜻이고 010-1234-5678 같은 전화번호 모양에 맞는다.

09-05 전처리 실습

1. O
2. X — <UNK>(1번)로 바꿉니다.
3. ②
4. ①
5. UNK
6. 6 — 문장 6개, 길이 6

7. 예시 답: 단어 집합을 상위 10개로 잡았더니 "시간", "아깝다", "지루함"이 모두 <UNK>(1)가 되어 [1, 1, 2, 1]이 됐다. 부정적인 리뷰라는 단서가 사라지고 "영화"만 남았다. 단어 집합이 작으면 정보를 잃는다.

용어 정리

용어	정의
말뭉치 (corpus)	분석하려고 모아 둔 글 전체
토큰 / 토큰화	글을 나눈 단위 조각 / 글을 토큰으로 나누는 작업
어절 / 형태소	띄어쓰기 단위 / 뜻을 가진 가장 작은 말의 단위
교착어	단어 뒤에 조사·어미가 붙어 문법적 역할을 나타내는 언어 (한국어)
품사 태깅	토큰마다 명사, 조사 등 품사를 붙이는 것
정제 / 정규화	노이즈 데이터 제거 / 표기가 다른 같은 단어를 통합
불용어 (stopword)	자주 나오지만 분석에 도움이 되지 않는 단어
정규 표현식	문자열의 패턴을 기호로 적은 표기법 (파이썬 `re` 모듈)
단어 집합 (vocabulary)	서로 다른 토큰에 번호를 붙인 목록. 크기 V
정수 인코딩	토큰을 단어 집합의 번호로 바꾸는 것
OOV, <UNK>	단어 집합에 없는 단어 / 그 자리를 대신하는 토큰
패딩, <PAD>	문장 길이를 같게 맞추는 것 / 빈자리를 채우는 토큰